AI Skill Gap Intelligence Platform

Frontend Development Work Division — Member 1 & Member 2

Frontend direction: The team will build a premium, clean career-intelligence web application inspired by the approved reference design. The visual direction is **minimal on the landing page** and **information-rich on the results dashboard**. The application will connect the React frontend to the existing Flask + NLP + ML + PostgreSQL backend.

1. Approved Product & Design Direction

• **Product identity:** CareerGenome / Career Intelligence

• **Theme:** White/off-white interface with indigo, violet and teal accents; rounded cards, soft shadows, subtle gradients and clean typography.

• **Landing page:** Very minimal and spacious. Avoid crowding the first screen with charts, statistics or too many cards.

• **Analysis page:** Focused workflow — upload resume, select target role, start analysis, then show an animated processing state.

• **Results page:** Detailed dashboard inspired by the provided reference: skill match, skill gap, charts, Skill DNA, matching skills, skills to develop, job recommendations and career path.

• **Important:** Mockup values are visual examples only. Real values must come from the backend API.

2. Technology Stack

Technology	Purpose
React.js + Vite	Frontend application and development environment
Tailwind CSS	Responsive styling and theme system
Framer Motion	Page transitions, loading states and micro-animations
Lucide React	Consistent interface icons
Recharts	Skill distribution and Skill DNA visualizations
Axios	Communication with the Flask backend API
Flask API	Existing backend endpoint for resume analysis

3. MEMBER 1 — UI/UX, LANDING & ANALYZE PAGE

Main responsibility: Build the visual foundation and user-facing workflow before results are displayed. Member 1 owns the shared design system, navigation, landing page, resume-analysis page and analysis/loading experience.

A. Project & Design Foundation

• Create the React + Vite frontend project.

• Install and configure Tailwind CSS, Framer Motion, Lucide React, Recharts and Axios.

• Create the common theme: off-white background, deep indigo primary, violet secondary, teal accent, white cards and dark text.

• Create reusable base components such as Button, Card, Navbar, Section, Badge and SkillChip where useful.

• Set up responsive behavior for desktop, tablet and mobile.

B. Navbar / Global Layout

• Create the main navigation: CareerGenome logo, Home, Analyze, About and Get Started.

• Keep the navbar consistent across all pages.

• Create active navigation states and mobile navigation if required.

• Create common page containers, spacing and typography rules.

C. Landing / Home Page

• Create a clean hero section with the headline: “Turn Your Resume Into Your Career Roadmap.”

• Add a short explanation of resume analysis, skill-gap detection and career guidance.

• Add the main CTA: “Analyze My Resume”.

• Add a secondary action such as “Watch Demo” only if useful.

• Create a subtle career/skills visual on the right side without overcrowding the page.

• Create a simple “How It Works” section with three stages: Upload Resume → AI Analysis → Get Insights.

• Add a final CTA section if desired, while preserving whitespace and minimalism.

• Do not use unverified statistics such as “10K+ resumes” or “95% accuracy” as real project claims.

D. Analyze / Resume Upload Page

• Create the upload workflow shown in the approved reference.

• Build drag-and-drop / file-select UI.

• Add selected-file state, file name and remove/reset behavior.

• Create target-role input or selector.

• Add popular role chips such as Data Analyst, Data Scientist, Business Analyst and others as UI options.

• Create the Analyze Resume button.

• Validate required inputs before submission.

• Keep the page visually simple and focused.

E. Analysis / Loading State

• Create the animated processing screen.

• Show steps such as Resume uploaded successfully, Extracting skills using NLP, Comparing with job requirements, Analyzing skill gaps and Generating recommendations.

• Use Framer Motion for subtle progress/check animations.

• The loading UI should be reusable by Member 2 when the real API call is running.

Member 1 Deliverables

Expected output: A polished Home page, Analyze page, shared theme/components, responsive navigation and reusable analysis-loading UI. Member 1 should not hardcode backend result data into the dashboard.

4. MEMBER 2 — API INTEGRATION & RESULTS DASHBOARD

Main responsibility: Connect the frontend to the existing Flask/NLP/ML pipeline and turn the returned analysis into the Career Intelligence dashboard. Member 2 owns the data flow, dashboard components, charts, recommendations and integration testing.

A. API Service Layer

• Create a central Axios service, for example src/services/api.js.

• Configure the Flask backend base URL in one place.

• Create the POST request for /api/analyze-resume.

• Send the resume file and target_role using multipart/form-data.

• Handle success, validation errors, backend errors and network failures.

• Do not expose database credentials or PostgreSQL connection details in frontend code.

B. Analysis State Management

• Connect Member 1’s Analyze form to the API service.

• Show the loading/analysis animation while the backend pipeline is running.

• Store the returned JSON analysis in frontend state.

• Handle empty or missing response fields safely.

• Navigate/show the Dashboard only after successful analysis.

• Preserve a clear error state if analysis fails.

C. Results Dashboard — Header & Match Summary

• Create “Your Career Intelligence” heading and supporting text.

• Show target role returned by the analysis.

• Create the main Skill Match percentage indicator.

• Create summary cards for total skills, matching skills, missing skills and skill gap.

• Use real API values rather than the visual mockup values.

D. Skill Distribution Chart

• Use Recharts to compare the student’s skills with relevant industry/job requirements.

• Create a clean bar chart similar to the reference.

• Make the chart responsive and readable.

• Ensure chart labels are generated from actual response data where available.

E. Skill DNA

• Create the signature Skill DNA radar chart using Recharts.

• Represent the student skill profile and required/target skill profile.

• Use a restrained number of dimensions so the radar chart stays readable.

• Use the approved purple/teal visual language.

F. Matching Skills

• Display matching skills returned by the backend as reusable skill chips.

• Show the total number of matching skills.

• Avoid hardcoding example skills once API integration is complete.

G. Skills to Develop / Skill Gap

• Display missing skills from the backend.

• Show priority or gap information only when the backend provides it.

• Use progress bars or priority labels for a polished visual.

• Clearly distinguish missing skills from skills the student already possesses.

H. Job Recommendations

• Display the top recommended jobs from the ML/recommendation pipeline.

• Show job title, match score and relevant matching/missing skill information when available.

• Create reusable recommendation cards.

• Sort/display according to the backend ranking rather than inventing frontend rankings.

I. Career Path

• Create a visual career journey section inspired by the reference.

• Show current role, next step and future opportunity when such information is available.

• Keep this component data-driven so it can be improved later.

J. Integration & Testing

• Test the complete flow: upload → target role → API → NLP/ML → JSON → dashboard.

• Test success, invalid file, missing target role, backend failure and network failure states.

• Test responsive dashboard layout.

• Check that charts do not break when arrays are empty or small.

• Coordinate with Member 1 before merging shared components.

Member 2 Deliverables

Expected output: Axios API layer, real backend integration, loading/error handling, complete Career Intelligence dashboard, charts, Skill DNA, skill-gap sections, job recommendations and end-to-end testing.

5. Shared Final Folder Structure

frontend/
■■■ public/
■■■ src/
■ ■■■ assets/
■ ■■■ components/
■ ■ ■■■ Navbar.jsx
■ ■ ■■■ Button.jsx
■ ■ ■■■ Card.jsx
■ ■ ■■■ SkillChip.jsx
■ ■ ■■■ LoadingAnalysis.jsx
■ ■■■ pages/
■ ■ ■■■ Home.jsx
■ ■ ■■■ Analyze.jsx
■ ■ ■■■ Dashboard.jsx
■ ■■■ charts/
■ ■ ■■■ SkillDistribution.jsx
■ ■ ■■■ SkillDNA.jsx
■ ■■■ services/
■ ■ ■■■ api.js
■ ■■■ App.jsx
■ ■■■ main.jsx
■ ■■■ index.css
■■■ package.json
■■■ vite.config.js

6. Git / Collaboration Strategy

• **Do not work directly on the same branch.**

• Member 1: create/use a dedicated frontend UI branch such as **frontend-member1**.

• Member 2: create/use a dedicated integration branch such as **frontend-member2**.

• Both branches should start from the agreed project branch and should commit small, meaningful changes.

• After individual work is tested, merge the branches carefully. Resolve conflicts together, especially in App.jsx, shared components and package configuration.

• Do not force-push over the team's shared history.

7. Final End-to-End Flow

1. User opens CareerGenome → minimal landing page.
2. User clicks Analyze My Resume.
3. User uploads resume and selects target role.
4. Frontend sends the file + role to Flask.
5. Flask runs resume extraction, NLP skill matching, skill-gap analysis and recommendations.
6. Frontend shows the animated analysis state while processing.
7. Backend returns the final analysis JSON.
8. Dashboard renders real values: Skill Match, Skill Gap, Skill DNA, matching skills, missing skills, recommendations and career path.
9. User can understand exactly what skills to improve and which roles are better aligned.

8. Important Implementation Rules

• Keep the landing page minimal; reserve detailed visualizations for the results dashboard.

• Use the approved white + indigo + violet + teal theme consistently.

• Use real backend data wherever the backend already provides it.

• Do not present mockup numbers as actual project results.

• Do not claim PDF/DOCX resume support unless the backend is updated to support those formats. The current backend flow is based on TXT resumes.

• Do not put PostgreSQL credentials in React code.

• Keep components reusable so Member 1 and Member 2 can merge their work cleanly.

• Prioritize responsive design and readable charts over excessive animation.

Final goal: The frontend should feel like a real AI career product — minimal and welcoming at the start, focused during analysis, and rich with actionable intelligence after the analysis.